STAYKARO PRIVATE LIMITED

AI CALLER AGENT

Database Design & Data Model

Complete schema, ER diagrams, RLS, and migration/backup strategy

Version 1.1
PostgreSQL 15+ with pgvector
Companion to the Architecture Specification (§9-16) and API Specification

Table of Contents

1. What Is a Relational Database, and Why PostgreSQL?

CONCEPT · Relational database

WHAT IT IS

A relational database stores data in tables -- rows and columns, like a set of connected spreadsheets -- and lets you query across them using relationships (a foreign key in one table pointing at another table's primary key).

REAL-WORLD ANALOGY

Think of a well-run filing system with cross-references: a customer's folder doesn't repeat every order they've ever placed inside it -- it holds a customer ID, and the orders folder holds the same ID on each order. Look up the customer once, and every order that references them is instantly findable, with no duplicated, potentially-inconsistent copies of the customer's name scattered across a hundred order forms.

WHY STAYKARO USES IT

Almost everything permanent in this system -- tenants, agents, calls, invoices -- has clear relationships to other things (a call belongs to a tenant and an agent; an invoice belongs to a subscription). A relational database is built exactly for that shape of data, with strong guarantees that a foreign key can't silently point at nothing.

TECHNICAL IMPLEMENTATION

PostgreSQL specifically, because it's open-source, extremely mature, and -- critically for this project -- supports two extensions we rely on directly: **pgvector** (§3, for RAG) and native **Row-Level Security** (§9, for tenant isolation), without needing a second specialized database system bolted on.

2. Entity-Relationship Diagram: Identity & Agents

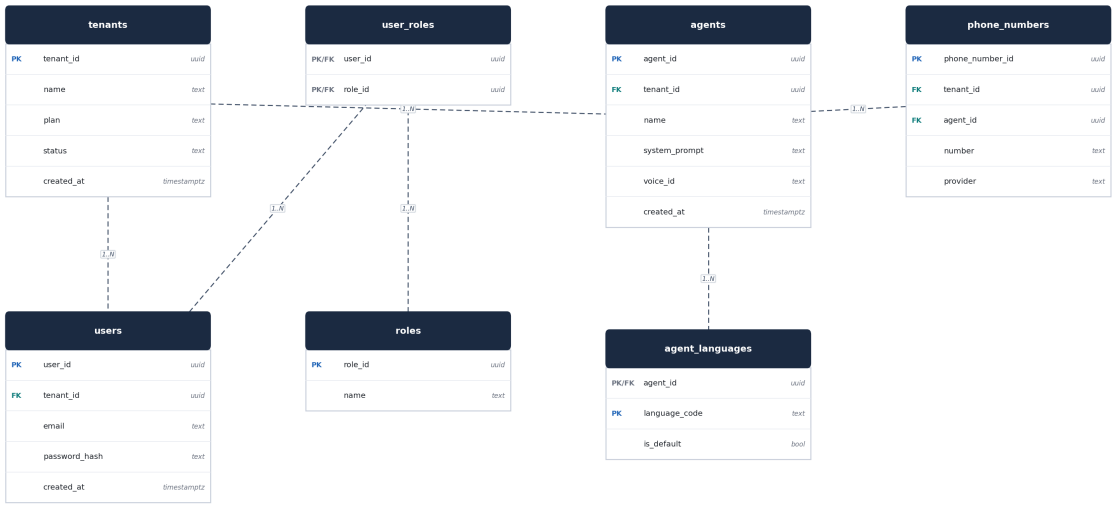

Figure 2.1 -- Identity and agent-configuration tables. Dashed lines show foreign-key relationships; PK = primary key, FK = foreign key.

tenants

Why it exists: The root record for every business running an agent on the platform.

Problem it solves: Everything else in the system needs a single, unambiguous "which business is this" anchor to hang off of.

Used by: Multi-tenant architecture (Architecture Spec §9) · **Owned by:** Nishkala

Column	Type	Key	Notes
tenant_id	uuid	PK	Referenced by nearly every other table
name	text		Display name, e.g. "ABC Motors"
plan	text		Current billing plan identifier
status	text		active / suspended
created_at	timestamptz		

users

Why it exists: Login accounts -- both Staykaro internal staff and client-side dashboard users.

Problem it solves: Authentication (Developer Handbook §15, API Spec §2) needs somewhere to check credentials against.

Used by: Auth service (packages/auth) · **Owned by:** Nishkala

Column	Type	Key	Notes
user_id	uuid	PK	
tenant_id	uuid	FK → tenants	NULL for Staykaro internal staff
email	text		Unique
password_hash	text		bcrypt -- never the plain password
created_at	timestamptz		

roles / user_roles

Why it exists: The fixed role vocabulary (Super Admin, Operations, Support, Client Admin, Client User) and the mapping of which users hold which roles.

Problem it solves: RBAC (Architecture Spec §26) needs role assignment to be data, not something hardcoded per user in application code.

Used by: Auth service (packages/auth) · **Owned by:** Nishkala

Column	Type	Key	Notes
role_id	uuid	PK	
name	text		One of the five fixed roles
user_id	uuid	PK/FK (user_roles)	Composite key with role_id
role_id	uuid	PK/FK (user_roles)	

agents

Why it exists: One row per configured AI agent -- a tenant's phone-answering persona, prompt, and voice.

Problem it solves: A tenant can eventually run more than one agent (e.g. sales vs. support); this table is what the Client Dashboard's agent-config screen (Execution Plan NH-09) reads and writes.

Used by: Voice gateway (reads at call start), Client Dashboard (config) · **Owned by:** Nishkala

Column	Type	Key	Notes
agent_id	uuid	PK	
tenant_id	uuid	FK → tenants	
name	text		Internal label
system_prompt	text		
voice_id	text		provider:voice_id pair, e.g. cartesia:hi-warm-1
created_at	timestamptz		

agent_languages

Why it exists: Which languages an agent supports, and which is the default.

Problem it solves: The Architecture Spec's multilingual requirement (§13) needs this to be per-agent configuration, not a global constant.

Used by: Voice gateway (SH-12), Client Dashboard · **Owned by:** Nishkala

Column	Type	Key	Notes
agent_id	uuid	PK/FK → agents	
language_code	text	PK	e.g. te-IN
is_default	bool		Exactly one true per agent

phone_numbers

Why it exists: The mapping from a real phone number to the tenant and agent that owns it.

Problem it solves: The very first thing that happens on an inbound call (Architecture Spec §7) is resolving a dialed number to a tenant_id and agent_id -- this table is that lookup.

Used by: Voice gateway (call routing, first lookup on every call) · **Owned by:** Nishkala

Column	Type	Key	Notes
phone_number_id	uuid	PK	
tenant_id	uuid	FK → tenants	
agent_id	uuid	FK → agents	
number	text		E.164 format, unique
provider	text		exotel / twilio

3. Entity-Relationship Diagram: Knowledge & Calls

CONCEPT · pgvector, and why we store embeddings at all

WHAT IT IS

pgvector is a PostgreSQL extension that adds a new column type -- `vector` -- and the ability to search "which rows have a vector most similar to this one," directly inside PostgreSQL, using normal SQL.

REAL-WORLD ANALOGY

Imagine a library where books aren't just alphabetized by title, but where you could also ask "bring me the books that feel most similar in topic to this one I'm holding," and a librarian who has genuinely read every book instantly hands you the closest matches -- even if none of them share a single word with the book in your hand.

WHY STAYKARO USES IT

A caller might ask "how much does it cost" while the uploaded document says "pricing starts at...". An exact keyword search would miss that; a similarity search over embeddings (Architecture Spec §15) finds it because the two phrases mean similar things, even with different words -- and this matters even more across languages and code-mixed speech, per the multilingual retrieval requirement in §16.

TECHNICAL IMPLEMENTATION

Each row in `knowledge_chunks` stores its text alongside a `vector(1536)` embedding. A caller's question is embedded the same way at query time, and pgvector's <=> operator finds the closest matches, filtered first by `tenant_id`.

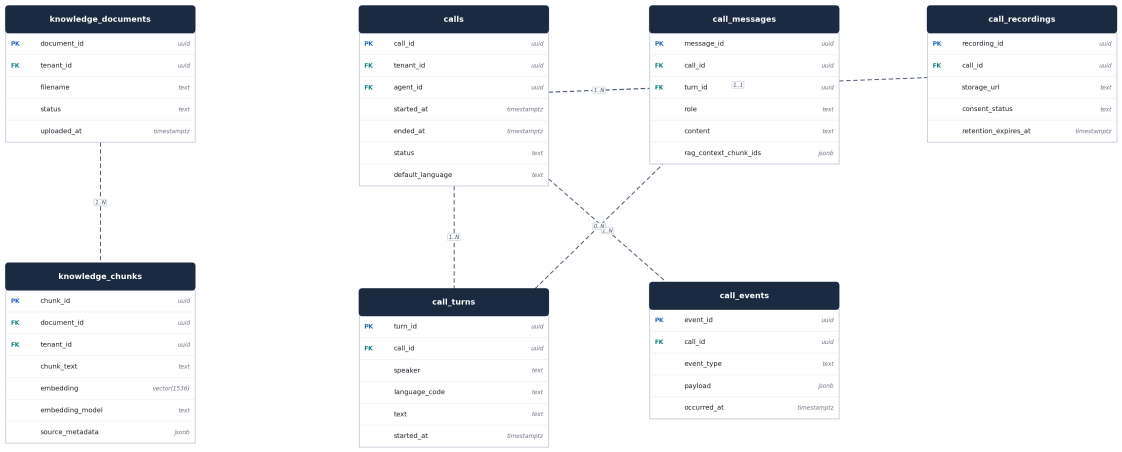

Figure 3.1 -- Knowledge base and call-record tables.

knowledge_documents

Why it exists: One row per file a client uploads to their agent's knowledge base.

Problem it solves: Tracks processing status end-to-end (Architecture Spec §14) so the Client Dashboard can show "processing → ready" rather than the upload just silently disappearing for a while.

Used by: Ingestion worker (NK-09), Client Dashboard (NH-10) · **Owned by:** Nishkala

Column	Type	Key	Notes
document_id	uuid	PK	
tenant_id	uuid	FK → tenants	
filename	text		
status	text		processing / ready / failed
uploaded_at	timestamptz		

knowledge_chunks

Why it exists: One row per chunk of extracted, embedded text from a knowledge document.

Problem it solves: RAG retrieval (Architecture Spec §15) searches this table directly -- it's the table pgvector's similarity search actually runs against.

Used by: RAG retrieval service (NK-10) · **Owned by:** Nishkala

Column	Type	Key	Notes
chunk_id	uuid	PK	
document_id	uuid	FK → knowledge_documents	
tenant_id	uuid	FK → tenants	Denormalized here on purpose -- see note below
chunk_text	text		
embedding	vector(1536)		Indexed -- see §8
embedding_model	text		So a model change can be identified and re-indexed
source_metadata	jsonb		Which document/page this came from, for traceability

WHY tenant_id IS DUPLICATED ONTO knowledge_chunks

knowledge_chunks could look up its tenant through document_id → knowledge_documents → tenant_id. We store tenant_id directly on the chunk instead, because it's the single most latency- and security-critical filter in the whole system (Architecture Spec §10, §15) -- every live retrieval query filters on it, on every turn, of every call. A direct column means a straightforward index and a straightforward RLS policy, instead of a join sitting on the critical path of every single conversational turn.

calls

Why it exists: One row per phone call, created the moment a call connects.

Problem it solves: The anchor record every other call-related table (turns, messages, events, recordings, usage) hangs off.

Used by: Voice gateway (SH-11, internal API), both dashboards (read) · **Owned by:** Nishkala

Column	Type	Key	Notes
call_id	uuid	PK	Generated by the voice gateway
tenant_id	uuid	FK → tenants	
agent_id	uuid	FK → agents	
started_at	timestamptz		
ended_at	timestamptz		NULL while call is active
status	text		active / completed / failed
default_language	text		Starting language; see call_turns for per-turn language

call_turns

Why it exists: One row per conversational turn -- one caller utterance or one agent response.

Problem it solves: This is where the turn-level language_code requirement (Architecture Spec §8, §13) actually lives as data -- the thing that makes mid-call language switching representable at all.

Used by: Voice gateway (SH-11), transcript API (API Spec §6) · **Owned by:** Nishkala

Column	Type	Key	Notes
turn_id	uuid	PK	
call_id	uuid	FK → calls	
speaker	text		caller / agent
language_code	text		Per-turn, not per-call
text	text		Transcribed or generated text for this turn
started_at	timestamptz		

call_messages

Why it exists: One row per message exchanged with the LLM -- system prompt, retrieved context, user message, assistant response.

Problem it solves: Distinct from call_turns: a single conversational turn can involve multiple LLM-level messages (e.g. the injected RAG context). This table is what you'd inspect to debug "why did the agent say that," not just "what did the agent say."

Used by: RAG/LLM integration (SH-14), debugging/support · **Owned by:** Nishkala

Column	Type	Key	Notes
message_id	uuid	PK	
call_id	uuid	FK → calls	
turn_id	uuid	FK → call_turns	NULL for e.g. the system prompt message
role	text		system / user / assistant
content	text		
rag_context_chunk_ids	jsonb		Which knowledge_chunks were injected, if any

call_events

Why it exists: One row per notable event during a call that isn't a spoken turn -- a barge-in, a provider fallback, an error.

Problem it solves: This is the data behind the Failure Architecture (Architecture Spec §18) actually being verifiable after the fact, not just something that happened and left no trace.

Used by: Voice gateway (SH-16), Admin live-call view (NH-07) · **Owned by:** Nishkala

Column	Type	Key	Notes
event_id	uuid	PK	
call_id	uuid	FK → calls	
event_type	text		barge_in / provider_fallback / stt_failure / etc.
payload	jsonb		Event-specific detail
occurred_at	timestamptz		

call_recordings

Why it exists: One row per call recording, where recording is legally permitted and consented to.

Problem it solves: Recording access needs its own consent/retention tracking, distinct from the transcript, per the Architecture Spec's compliance model (§28).

Used by: Voice gateway (writes), Client/Admin dashboards (read, access-logged) · **Owned by:** Nishkala

Column	Type	Key	Notes
recording_id	uuid	PK	
call_id	uuid	FK → calls	
storage_url	text		
consent_status	text		recorded / declined / not_applicable
retention_expires_at	timestamptz		Enforces the retention policy from Architecture Spec §28

4. Entity-Relationship Diagram: Usage & Billing

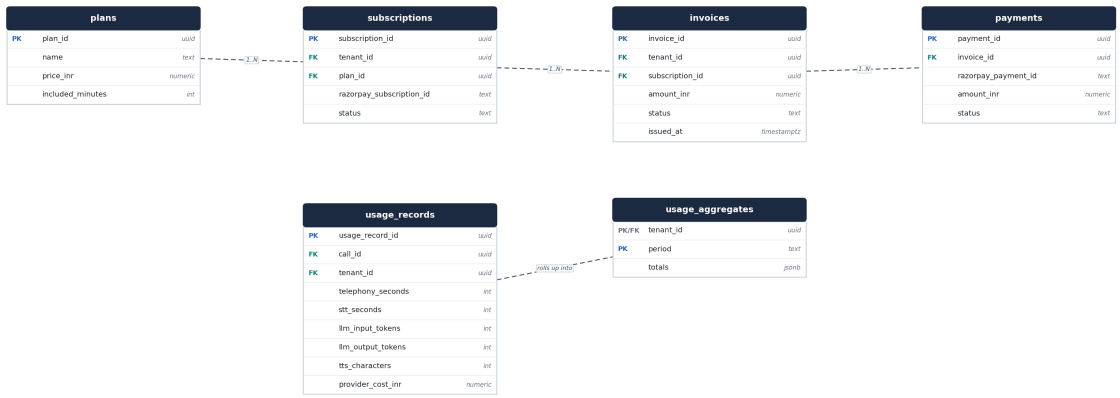

Figure 4.1 -- Usage metering and billing tables.

usage_records

Why it exists: One row per call, with granular resource consumption -- exactly the fields the Architecture Spec's usage-metering requirement (§21) lists.

Problem it solves: Without this, margin can only be estimated from call duration, which is precisely the mistake the architecture review flagged as a risk (a ■999 plan silently costing ■1,340 to serve).

Used by: Usage metering service (NK-13), Admin cost screen (NH-08) · **Owned by:** Nishkala

Column	Type	Key	Notes
usage_record_id	uuid	PK	
call_id	uuid	FK → calls	
tenant_id	uuid	FK → tenants	
telephony_seconds	int		
stt_seconds	int		
llm_input_tokens	int		
llm_output_tokens	int		
tts_characters	int		
provider_cost_inr	numeric		Computed from provider pricing at time of call

usage_aggregates

Why it exists: Pre-computed monthly totals per tenant, rolled up from usage_records.

Problem it solves: The usage-summary endpoint (API Spec §7) and billing period close both need fast month-level totals without re-summing potentially thousands of call-level rows on every dashboard load.

Used by: Usage metering service (NK-13) · **Owned by:** Nishkala

Column	Type	Key	Notes
tenant_id	uuid	PK/FK → tenants	
period	text	PK	YYYY-MM
totals	jsonb		Same fields as usage_records, summed

plans / subscriptions / invoices / payments

Why it exists: The Razorpay-backed billing chain: a plan a tenant subscribes to, the invoices that subscription generates, and the payments against those invoices.

Problem it solves: Mirrors the billing flow in Architecture Spec §22 as data, so billing state can be reconstructed and reconciled independently of what Razorpay's dashboard shows.

Used by: Billing backend (NK-14, NK-15), Client billing screen (NH-12) · **Owned by:** Nishkala

Column	Type	Key	Notes
plan_id / subscription_id / invoice_id / payment_id	uuid	PK	One per table
tenant_id	uuid	FK → tenants	On subscriptions and invoices
razorpay_subscription_id / razorpay_payment_id	text		External reference for reconciliation
status	text		Mirrors Razorpay's own status vocabulary

billing_events

Why it exists: One row per processed Razorpay webhook event, keyed by Razorpay's own event ID.

Problem it solves: This is what makes webhook handling idempotent (Architecture Spec §22, API Spec §7) -- a duplicate delivery is detected here before any billing state is touched twice.

Used by: Webhook handler (NK-15) · **Owned by:** Nishkala

Column	Type	Key	Notes
event_id	uuid	PK	
razorpay_event_id	text	Unique	The idempotency key
event_type	text		
processed_at	timestamptz		

5. Operations Tables

audit_logs

Why it exists: One row per sensitive action -- role changes, cross-tenant access attempts, billing changes, data exports, admin actions on a client account.

Problem it solves: When something disputed happens ("who changed our plan"), this is the record that answers it -- required by the Security Architecture, Architecture Spec §27.

Used by: packages/observability (NK-16) · **Owned by:** Nishkala

Column	Type	Key	Notes
audit_id	uuid	PK	
tenant_id	uuid	FK → tenants	NULL for platform-level events
user_id	uuid	FK → users	
action	text		
before_value / after_value	jsonb		
occurred_at	timestamptz		

integration_events

Why it exists: One row per post-call automation event -- the async trigger for CRM/WhatsApp/email (Architecture Spec §23).

Problem it solves: Decouples the live call from automation entirely: this table is what the integration service polls or consumes, so a slow or failed automation can never reach back and affect a call.

Used by: Integration service (NH-13) · **Owned by:** Nihal (via Nishkala's schema)

Column	Type	Key	Notes
event_id	uuid	PK	
call_id	uuid	FK → calls	
event_type	text		call_completed, etc.
status	text		pending / processed / failed
retry_count	int		

background_jobs

Why it exists: General-purpose job queue table for asynchronous work that isn't automation -- document processing, usage aggregation, retries.

Problem it solves: Some background work (ingestion, usage rollups) doesn't fit "integration event," but still needs a durable record that survives a worker restart.

Used by: Background workers (NK-09, NK-13) · **Owned by:** Nishkala

Column	Type	Key	Notes
job_id	uuid	PK	
job_type	text		ingestion / usage_rollup / etc.
status	text		queued / running / done / failed
payload	jsonb		
scheduled_at	timestamptz		

provider_events

Why it exists: One row per notable external-provider event -- a failure, a fallback trigger, a rate-limit hit.

Problem it solves: Aggregated separately from call_events because these matter at the platform level ("is Groq degraded right now, across all tenants"), not just the single-call level.

Used by: Provider abstraction layer (SH-07, SH-09, SH-16) · **Owned by:** Nishkala

Column	Type	Key	Notes
event_id	uuid	PK	
provider	text		deepgram / groq / openai / cartesia / etc.
event_type	text		failure / fallback_triggered / rate_limited
call_id	uuid	FK → calls	NULL if not tied to a specific call
occurred_at	timestamptz		

6. PostgreSQL Row-Level Security (RLS)

CONCEPT · Row-Level Security

WHAT IT IS

RLS is a PostgreSQL feature that attaches a filtering rule directly to a table, at the database level -- so that even a query which forgot to add `WHERE tenant_id = ...` still cannot see another tenant's rows.

REAL-WORLD ANALOGY

Application-level filtering is like trusting every waiter to only bring food from the table that ordered it. RLS is the kitchen itself refusing to hand over a dish to anyone who can't prove which table it's for, regardless of which waiter is asking.

WHY STAYKARO USES IT

The Architecture Specification is explicit (§10) that tenant isolation cannot depend on application code remembering to filter correctly, every time, forever. RLS is the second, independent layer that holds even if a future developer -- or a future AI-generated PR -- forgets a WHERE clause.

TECHNICAL IMPLEMENTATION

Every tenant-scoped table has a policy that compares its `tenant_id` column against a per-transaction session setting, set fresh on every request (see below).

```
-- Enable RLS on a tenant-scoped table
ALTER TABLE knowledge_chunks ENABLE ROW LEVEL SECURITY;

-- Policy: a row is visible only if its tenant_id matches
-- the tenant_id set for the current transaction
CREATE POLICY tenant_isolation ON knowledge_chunks
    USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Set at the START of every request-scoped transaction --
-- never at the connection level (see the note below)
SET LOCAL app.current_tenant = '123e4567-e89b-12d3-a456-426614174000';
```

Figure 6.1 -- RLS policy and the per-transaction context it depends on

THE ONE MISTAKE THAT MATTERS HERE

`SET LOCAL` scopes the setting to the current transaction only -- it automatically resets when the transaction ends. If tenant context were set at the connection level instead (`SET` without `LOCAL`), a pooled connection returned to the pool and reused by a different tenant's request could carry the previous tenant's context with it. Architecture Spec §10 and Execution Plan ticket NK-07 both call this out as the highest-stakes review in the whole system for exactly this reason.

7. Constraints

- **NOT NULL** on every foreign key that must always resolve (e.g. call_turns.call_id) -- a turn with no call is a data-integrity bug, not a valid edge case.
- **UNIQUE** on phone_numbers.number, users.email, billing_events.razorpay_event_id -- the last one is what makes webhook idempotency (§5 above) enforceable at the database level, not just in application logic.

- **CHECK** constraints on status-like columns (e.g. calls.status IN ('active','completed','failed')) so an invalid status can't be written even by a bug that bypasses application-level validation.
- **ON DELETE CASCADE** from knowledge_documents to knowledge_chunks -- deleting a document must not leave orphaned, still-searchable embeddings pointing at text that officially no longer exists (API Spec §5).

8. Indexes

CONCEPT · Why indexes matter

WHAT IT IS

An index is a separate, pre-sorted structure PostgreSQL maintains alongside a table, so it can find matching rows without scanning every row in the table.

REAL-WORLD ANALOGY

It's the difference between using a book's index to jump straight to page 214, versus reading every page from the start until you happen to find what you're looking for.

WHY STAYKARO USES IT

Every tenant-scoped query filters by tenant_id, on every request, many times per call. Without an index, that filter still works correctly -- it's just a full table scan every time, which is exactly the kind of avoidable delay the latency budget in Architecture Spec §20 has no room for.

TECHNICAL IMPLEMENTATION

Key indexes: a B-tree index on every tenant_id column used in a WHERE clause; a composite index on (tenant_id, call_id) for call_turns and call_messages; and, most importantly for RAG performance, an HNSW index on knowledge_chunks.embedding for fast approximate nearest-neighbor search.

```
CREATE INDEX idx_chunks_tenant ON knowledge_chunks (tenant_id);
CREATE INDEX idx_chunks_embedding ON knowledge_chunks
    USING hnsw (embedding vector_cosine_ops);
CREATE INDEX idx_calls_tenant_started ON calls (tenant_id, started_at DESC);
```

Figure 8.1 -- Representative index definitions

9. Migration Strategy

Schema changes are managed with Alembic (Developer Handbook §5.2), never by hand-editing the production database:

1. Every schema change is a new, timestamped Alembic migration file, committed alongside the code that needs it.
2. Migrations run automatically in CI against a throwaway test database (Developer Handbook §11) before a PR can merge, catching a broken migration before it ever reaches staging.
3. Migrations run in staging, verified, before running in production -- never the reverse.
4. Every migration must have a working `downgrade()`, even if it's rarely used -- an un-reversible migration is a one-way door applied to a live production database.

10. Backup & Restore Strategy

CONCEPT · Why a backup that's never been restored doesn't count as a backup

WHAT IT IS

A backup is a saved copy of the database that can be used to recover if the live one is lost or corrupted. A restore is the process of actually using that copy to bring a working database back.

REAL-WORLD ANALOGY

A fire extinguisher nobody has ever tested might be empty, expired, or simply the wrong type for the fire you eventually have. You don't find out until the moment you needed it to work.

WHY STAYKARO USES IT

This system holds paying clients' billing records and every caller's conversation history. "We take backups" is not the same claim as "we've proven we can get back to a working state from one," and the Architecture Spec (§33) is explicit that only the second claim counts.

TECHNICAL IMPLEMENTATION

Automated daily PostgreSQL backups (point-in-time recovery enabled), stored separately from the production VPS. Execution Plan ticket NK-17 requires an actual restoration -- to a clean environment, with data-integrity verification -- performed and documented before production launch, not merely scripted and assumed to work.

What's backed up	Frequency	Retention
PostgreSQL (full + point-in-time)	Daily full, continuous WAL	30 days
Knowledge document originals (object storage)	On upload	Matches document retention policy
Configuration / infrastructure-as-code	On every change (it's in Git)	Full history